

WEEK-2(Examples)

J.Simran

2520030166

Step 1: Create Database

The screenshot shows the SQL Developer interface with the 'Week_2_05*' script editor. The script contains two lines of SQL code:

```
1 • CREATE DATABASE IF NOT EXISTS hospital_db;
2 • USE hospital_db;
```

The 'Output' pane shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	16:50:11	CREATE DATABASE IF NOT EXISTS hospital_db	1 row(s) affected, 1 warning(s): 1007 Can't create database '...	0.000 sec
2	16:50:11	USE hospital_db	0 row(s) affected	0.000 sec

Step 2: Create physician Table

The screenshot shows the SQL Developer interface with the 'Week_2_05*' script editor. The script contains five lines of SQL code:

```
1 • CREATE DATABASE IF NOT EXISTS hospital_db;
2 • USE hospital_db;
3 • CREATE TABLE physician (
4     employeeid INT PRIMARY KEY,
5     name VARCHAR(100),
6     position VARCHAR(100),
7     ssn VARCHAR(20)
8 );
```

The 'Output' pane shows the execution results:

#	Time	Action	Message
1	16:50:11	CREATE DATABASE IF NOT EXISTS hospital_db	1 row(s) affected, 1 warning(s): 1007 Can't create database '...
2	16:50:11	USE hospital_db	0 row(s) affected
3	16:50:59	CREATE DATABASE IF NOT EXISTS hospital_db	1 row(s) affected, 1 warning(s): 1007 Can't create database '...
4	16:50:59	USE hospital_db	0 row(s) affected
5	16:50:59	CREATE TABLE physician (employeeid INT PRIMARY K...	0 row(s) affected

Step 3: Insert Physician Data

The screenshot shows the SQL Enterprise Manager interface with a query window titled 'Week_2_05'. The query is as follows:

```
10 (1,'John Dorian','Staff Internist','11111111'),
11 (2,'Elliot Reid','Attending Physician','22222222'),
12 (3,'Christopher Turk','Surgical Attending Physician','33333333'),
13 (4,'Percival Cox','Senior Attending Physician','44444444'),
14 (5,'Bob Kelso','Head Chief of Medicine','55555555'),
15 (6,'Todd Quinlan','Surgical Attending Physician','66666666'),
16 (7,'John Wen','Surgical Attending Physician','77777777'),
17 (8,'Keith Dudemeister','MD Resident','88888888'),
18 (9,'Molly Clock','Attending Psychiatrist','99999999');
19 SELECT * FROM physician;
```

The 'Result Grid' shows the data inserted into the 'physician' table:

employeeid	name	position	ssn
1	John Dorian	Staff Internist	11111111
2	Elliot Reid	Attending Physician	22222222
3	Christopher Turk	Surgical Attending Physician	33333333
4	Percival Cox	Senior Attending Physician	44444444
5	Bob Kelso	Head Chief of Medicine	55555555
6	Todd Quinlan	Surgical Attending Physician	66666666
7	John Wen	Surgical Attending Physician	77777777
8	Keith Dudemeister	MD Resident	88888888

The 'Output' pane shows the execution results:

#	Time	Action	Message
9	16:51:46	INSERT INTO physician VALUES (1,'John Dorian','Staff I...	9 row(s) affected Records: 9 Duplicates: 0 Warnings: 0
10	16:52:00	CREATE DATABASE IF NOT EXISTS hospital_db	1 row(s) affected, 1 warning(s): 1007 Can't create databas...
11	16:52:00	USE hospital_db	0 row(s) affected
12	16:52:00	CREATE TABLE physician (employeeid INT PRIMAR...	Error Code: 1050. Table 'physician' already exists
13	16:52:00	INSERT INTO physician VALUES (1,'John Dorian','Staff I...	Error Code: 1062. Duplicate entry '1' for key 'physician.PR...
14	16:52:00	SELECT * FROM physician LIMIT 0, 1000	9 row(s) returned

Step 4: Create department Table

The screenshot shows the SQL Enterprise Manager interface with a query window titled 'Week_2_05'. The query is as follows:

```
16 (7,'John Wen','Surgical Attending Physician','77777777'),
17 (8,'Keith Dudemeister','MD Resident','88888888'),
18 (9,'Molly Clock','Attending Psychiatrist','99999999');
19 SELECT * FROM physician;
20 CREATE TABLE department (
21     departmentid INT PRIMARY KEY,
22     name VARCHAR(100),
23     head INT
24 );
```

The 'Result Grid' shows the data inserted into the 'physician' table:

employeeid	name	position	ssn
1	John Dorian	Staff Internist	11111111
2	Elliot Reid	Attending Physician	22222222
3	Christopher Turk	Surgical Attending Physician	33333333
4	Percival Cox	Senior Attending Physician	44444444
5	Bob Kelso	Head Chief of Medicine	55555555
6	Todd Quinlan	Surgical Attending Physician	66666666
7	John Wen	Surgical Attending Physician	77777777
8	Keith Dudemeister	MD Resident	88888888
9	Molly Clock	Attending Psychiatrist	99999999

The 'Output' pane shows the execution results:

#	Time	Action	Message
15	16:52:46	CREATE DATABASE IF NOT EXISTS hospital_db	1 row(s) affected, 1 warning(s): 1007 Can't create databas...
16	16:52:47	USE hospital_db	0 row(s) affected
17	16:52:47	CREATE TABLE physician (employeeid INT PRIMAR...	Error Code: 1050. Table 'physician' already exists
18	16:52:47	INSERT INTO physician VALUES (1,'John Dorian','Staff I...	Error Code: 1062. Duplicate entry '1' for key 'physician.PR...

Step 5: Insert Department Data

SQL Script:

```
22     name VARCHAR(100),
23     head INT
24 );
25 • INSERT INTO department VALUES
26 (1,'General Medicine',4),
27 (2,'Surgery',7),
28 (3,'Psychiatry',9);
29 • SELECT * FROM department;
```

Result Grid:

departmentid	name	head
1	General Medicine	4
2	Surgery	7
3	Psychiatry	9
NULL	NULL	NULL

Action Output:

#	Time	Action	Message
26	16:53:30	CREATE TABLE department (departmentid INT PRIM...	Error Code: 1050. Table 'department' already exists
27	16:53:30	INSERT INTO department VALUES (1,'General Medicine'...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
28	16:53:56	CREATE DATABASE IF NOT EXISTS hospital_db	1 row(s) affected, 1 warning(s): 1007 Can't create databas...
29	16:53:56	USE hospital_db	0 row(s) affected
30	16:53:56	CREATE TABLE physician (employeeid INT PRIMAR...	Error Code: 1050. Table 'physician' already exists
31	16:53:56	INSERT INTO physician VALUES (1,'John Dorian','Staff I...	Error Code: 1062. Duplicate entry '1' for key 'physician.PR...
32	16:53:56	SELECT * FROM physician LIMIT 0, 1000	9 row(s) returned
33	16:53:56	CREATE TABLE department (departmentid INT PRIM...	Error Code: 1050. Table 'department' already exists
34	16:53:56	INSERT INTO department VALUES (1,'General Medicine'...	Error Code: 1062. Duplicate entry '1' for key 'department.P...
35	16:53:56	SELECT * FROM department LIMIT 0, 1000	3 row(s) returned

Step 6: Department + Head Physician

SQL Script:

```
27 (2,'Surgery',7),
28 (3,'Psychiatry',9);
29 • SELECT * FROM department;
30 • SELECT d.name AS Department,
31     p.name AS Head_Physician
32 FROM department d
33 JOIN physician p
34 ON d.head = p.employeeid;
```

Result Grid:

Department	Head_Physician
General Medicine	Percival Cox
Surgery	John Wen
Psychiatry	Molly Clock

Action Output:

#	Time	Action	Message
35	16:53:56	SELECT * FROM department LIMIT 0, 1000	3 row(s) returned
36	16:54:46	CREATE DATABASE IF NOT EXISTS hospital_db	1 row(s) affected, 1 warning(s): 1007 Can't create databas...
37	16:54:46	USE hospital_db	0 row(s) affected
38	16:54:46	CREATE TABLE physician (employeeid INT PRIMAR...	Error Code: 1050. Table 'physician' already exists
39	16:54:46	INSERT INTO physician VALUES (1,'John Dorian','Staff I...	Error Code: 1062. Duplicate entry '1' for key 'physician.PR...
40	16:54:46	SELECT * FROM physician LIMIT 0, 1000	9 row(s) returned
41	16:54:46	CREATE TABLE department (departmentid INT PRIM...	Error Code: 1050. Table 'department' already exists
42	16:54:46	INSERT INTO department VALUES (1,'General Medicine'...	Error Code: 1062. Duplicate entry '1' for key 'department.P...
43	16:54:46	SELECT * FROM department LIMIT 0, 1000	3 row(s) returned
44	16:54:46	SELECT d.name AS Department, p.name AS Head_...	3 row(s) returned

Step 7: Surgical Attending Physicians

The screenshot shows the SQL Developer interface with a query window titled 'Week_2_05'. The query is as follows:

```
30 SELECT d.name AS Department,
31        p.name AS Head_Physician
32 FROM department d
33 JOIN physician p
34 ON d.head = p.employeeid;
35 SELECT *
36 FROM physician
37 WHERE position = 'Surgical Attending Physician';
```

The 'Result Grid' shows the following data:

employeeid	name	position	ssn
3	Christopher Turk	Surgical Attending Physician	333333333
6	Todd Quinlan	Surgical Attending Physician	666666666
7	John Wen	Surgical Attending Physician	777777777
NULL	NULL	NULL	NULL

The 'Output' pane shows the execution log with the following messages:

#	Time	Action	Message
45	16:55:22	CREATE DATABASE IF NOT EXISTS hospital_db	1 row(s) affected, 1 warning(s): 1007 Can't create databas...
46	16:55:22	USE hospital_db	0 row(s) affected
47	16:55:22	CREATE TABLE physician (employeeid INT PRIMAR...	Error Code: 1050. Table 'physician' already exists
48	16:55:22	INSERT INTO physician VALUES (1,'John Dorian','Staff I...	Error Code: 1062. Duplicate entry '1' for key 'physician.PR...
49	16:55:22	SELECT * FROM physician LIMIT 0, 1000	9 row(s) returned
50	16:55:23	CREATE TABLE department (departmentid INT PRIM...	Error Code: 1050. Table 'department' already exists
51	16:55:23	INSERT INTO department VALUES (1,'General Medicine'...	Error Code: 1062. Duplicate entry '1' for key 'department.P...
52	16:55:23	SELECT * FROM department LIMIT 0, 1000	3 row(s) returned
53	16:55:23	SELECT d.name AS Department, p.name AS Head_...	3 row(s) returned
54	16:55:23	SELECT * FROM physician WHERE position = 'Surgical ...	3 row(s) returned

Step 9: Count Physicians

The screenshot shows the SQL Developer interface with a query window titled 'Week_2_05'. The query is as follows:

```
32 FROM department d
33 JOIN physician p
34 ON d.head = p.employeeid;
35 SELECT *
36 FROM physician
37 WHERE position = 'Surgical Attending Physician';
38 SELECT COUNT(*) AS Total_Physicians
39 FROM physician;
```

The 'Result Grid' shows the following data:

Total_Physicians
9

The 'Output' pane shows the execution log with the following messages:

#	Time	Action	Message
56	16:55:58	USE hospital_db	0 row(s) affected
57	16:55:58	CREATE TABLE physician (employeeid INT PRIMAR...	Error Code: 1050. Table 'physician' already exists
58	16:55:58	INSERT INTO physician VALUES (1,'John Dorian','Staff I...	Error Code: 1062. Duplicate entry '1' for key 'physician.PR...
59	16:55:58	SELECT * FROM physician LIMIT 0, 1000	9 row(s) returned
60	16:55:58	CREATE TABLE department (departmentid INT PRIM...	Error Code: 1050. Table 'department' already exists
61	16:55:58	INSERT INTO department VALUES (1,'General Medicine'...	Error Code: 1062. Duplicate entry '1' for key 'department.P...
62	16:55:58	SELECT * FROM department LIMIT 0, 1000	3 row(s) returned
63	16:55:58	SELECT d.name AS Department, p.name AS Head_...	3 row(s) returned
64	16:55:58	SELECT * FROM physician WHERE position = 'Surgical ...	3 row(s) returned
65	16:55:58	SELECT COUNT(*) AS Total_Physicians FROM physician...	1 row(s) returned

Step 10: Count Different Positions

SQL Developer interface showing the query execution for Step 10. The query window displays the following SQL script:

```
34 ON d.head = p.employeeid;
35 SELECT *
36 FROM physician
37 WHERE position = 'Surgical Attending Physician';
38 SELECT COUNT(*) AS Total_Physicians
39 FROM physician;
40 SELECT COUNT(DISTINCT position) AS Different_Positions
41 FROM physician;
```

The result grid shows the output of the query, displaying a single row with the value 7 for the 'Different_Positions' column.

The output window shows the execution log, including errors for table creation and successful results for the queries.

Step 11: Employees in Each Position

SQL Developer interface showing the query execution for Step 11. The query window displays the following SQL script:

```
37 WHERE position = 'Surgical Attending Physician';
38 SELECT COUNT(*) AS Total_Physicians
39 FROM physician;
40 SELECT COUNT(DISTINCT position) AS Different_Positions
41 FROM physician;
42 SELECT position,
43 COUNT(*) AS Total_Employees
44 FROM physician
45 GROUP BY position;
```

The result grid shows the output of the query, displaying a table with two columns: 'position' and 'Total_Employees'.

position	Total_Employees
Staff Internist	1
Attending Physician	1
Surgical Attending Physician	3
Senior Attending Physician	1
Head Chief of Medicine	1
MD Resident	1
Attending Psychiatrist	1

The output window shows the execution log, including successful results for the queries.

Step 12: Positions With More Than One Employee

The screenshot shows the SQL Developer interface with a query window titled 'Week_2_05'. The query is as follows:

```

39 FROM physician;
40 • SELECT COUNT(DISTINCT position) AS Different_Positions
41 FROM physician;
42 • SELECT position,
43     COUNT(*) AS Total_Employees
44 FROM physician
45 GROUP BY position;
46 • SELECT position,
47     COUNT(*) AS Total_Employees
48 FROM physician
49 GROUP BY position
50 HAVING COUNT(*) > 1;

```

The 'Result Grid' shows the following data:

position	Total_Employees
Surgical Attending Physician	3

The 'Output' pane shows the execution log with the following entries:

#	Time	Action	Message
99	16:58:01	SELECT d.name AS Department, p.name AS Head...	3 row(s) returned
100	16:58:01	SELECT * FROM physician WHERE position = 'Surgical ...	3 row(s) returned
101	16:58:01	SELECT COUNT(*) AS Total_Physicians FROM physician...	1 row(s) returned
102	16:58:01	SELECT COUNT(DISTINCT position) AS Different_Positi...	1 row(s) returned
103	16:58:01	SELECT position, COUNT(*) AS Total_Employees FR...	7 row(s) returned
104	16:58:01	SELECT position, COUNT(*) AS Total_Employees FR...	1 row(s) returned

Step 13: Alphabetical Physician Directory

The screenshot shows the SQL Developer interface with a query window titled 'Week_2_05'. The query is as follows:

```

47 COUNT(*) AS Total_Employees
48 FROM physician
49 GROUP BY position
50 HAVING COUNT(*) > 1;
51 • SELECT *
52 FROM physician
53 ORDER BY name ASC;

```

The 'Result Grid' shows the following data:

employeeid	name	position	ssn
5	Bob Kelso	Head Chief of Medicine	555555555
3	Christopher Turk	Surgical Attending Physician	333333333
2	Elliot Reid	Attending Physician	222222222
1	John Dorian	Staff Internist	111111111
7	John Wen	Surgical Attending Physician	777777777
8	Keith Dudemeister	MD Resident	888888888
9	Molly Clock	Attending Psychiatrist	999999999
4	Percival Cox	Senior Attending Physician	444444444
6	Todd Quinlan	Surgical Attending Physician	666666666

The 'Output' pane shows the execution log with the following entries:

#	Time	Action	Message
114	16:58:37	SELECT * FROM physician WHERE position = 'Surgical ...	3 row(s) returned
115	16:58:37	SELECT COUNT(*) AS Total_Physicians FROM physician...	1 row(s) returned
116	16:58:37	SELECT COUNT(DISTINCT position) AS Different_Positi...	1 row(s) returned
117	16:58:37	SELECT position, COUNT(*) AS Total_Employees FR...	7 row(s) returned
118	16:58:37	SELECT position, COUNT(*) AS Total_Employees FR...	1 row(s) returned
119	16:58:37	SELECT * FROM physician ORDER BY name ASC LIMIT...	9 row(s) returned

Step 14: Employee ID Highest to Lowest

J. Simran-2520030166 - Warni... x

Edit View Query Database Server Tools Scripting Help

Week_2_05*

Limit to 1000 rows

50 **HAVING** COUNT(*) > 1;

51 • **SELECT** *

52 **FROM** physician

53 **ORDER BY** name **ASC**;

54 • **SELECT** *

55 **FROM** physician

56 **ORDER BY** employeeid **DESC**;

employeeid	name	position	ssn
9	Molly Clock	Attending Psychiatrist	999999999
8	Keith Dudemeister	MD Resident	888888888
7	John Wen	Surgical Attending Physician	777777777
6	Todd Quinlan	Surgical Attending Physician	666666666
5	Bob Kelso	Head Chief of Medicine	555555555
4	Percival Cox	Senior Attending Physician	444444444
3	Christopher Turk	Surgical Attending Physician	333333333
2	Elliot Reid	Attending Physician	222222222
1	John Dorian	Staff Internist	111111111
NULL	NULL	NULL	NULL

Result Grid

Filter Rows:

Edit: Export/Import:

Result Grid Form Editor Field Types

an 51 Result 52 Result 53 Result 54 Result 55 physician 56 Apply Revert

Output

Action Output

#	Time	Action	Message
✓ 130	16:59:19	SELECT COUNT(*) AS Total_Physicians FROM physician...	1 row(s) returned
✓ 131	16:59:19	SELECT COUNT(DISTINCT position) AS Different_Positi...	1 row(s) returned
✓ 132	16:59:19	SELECT position, COUNT(*) AS Total_Employees FR...	7 row(s) returned
✓ 133	16:59:19	SELECT position, COUNT(*) AS Total_Employees FR...	1 row(s) returned
✓ 134	16:59:19	SELECT * FROM physician ORDER BY name ASC LIMIT...	9 row(s) returned
✓ 135	16:59:19	SELECT * FROM physician ORDER BY employeeid DES...	9 row(s) returned

Info Session

Step 15: Department Heads Only

J. Simran-2520030166 - Warni... x

Edit View Query Database Server Tools Scripting Help

Week_2_05*

Limit to 1000 rows

52 **FROM** physician

53 **ORDER BY** name **ASC**;

54 • **SELECT** *

55 **FROM** physician

56 **ORDER BY** employeeid **DESC**;

57 • **SELECT** *

58 **FROM** physician

59 **WHERE** employeeid **IN**

60

61 (

62 **SELECT** head

63 **FROM** department

63)

employeeid	name	position	ssn
4	Percival Cox	Senior Attending Physician	444444444
7	John Wen	Surgical Attending Physician	777777777
9	Molly Clock	Attending Psychiatrist	999999999
NULL	NULL	NULL	NULL

Result Grid

Filter Rows:

Edit: Export/Import:

Result Grid

62 Result 63 Result 64 Result 65 physician 66 physician 67 Apply Revert Context

Output

Action Output

#	Time	Action	Message
✓ 147	17:00:00	SELECT COUNT(DISTINCT position) AS Different_Positi...	1 row(s) returned
✓ 148	17:00:00	SELECT position, COUNT(*) AS Total_Employees FR...	7 row(s) returned
✓ 149	17:00:00	SELECT position, COUNT(*) AS Total_Employees FR...	1 row(s) returned
✓ 150	17:00:00	SELECT * FROM physician ORDER BY name ASC LIMIT...	9 row(s) returned
✓ 151	17:00:00	SELECT * FROM physician ORDER BY employeeid DES...	9 row(s) returned
✓ 152	17:00:00	SELECT * FROM physician WHERE employeeid IN (S...	3 row(s) returned

Info Session

Step 16: Physicians Who Are Not Department Heads

The screenshot shows a SQL query in a query editor window titled "Week_2_05". The query is as follows:

```
62 FROM department
63 );
64 SELECT *
65 FROM physician
66 WHERE employeeid NOT IN
67 (
68     SELECT head
69     FROM department
70 );
```

The query results are displayed in a grid with the following columns: employeeid, name, position, and ssn. The results show 8 rows of data:

employeeid	name	position	ssn
1	John Dorian	Staff Internist	111111111
2	Elliot Reid	Attending Physician	222222222
3	Christopher Turk	Surgical Attending Physician	333333333
4	Bob Kelso	Head Chief of Medicine	555555555
5	Todd Quinlan	Surgical Attending Physician	666666666
6	Keith Dudemeister	MD Resident	888888888
7			
8			

The bottom of the screenshot shows the "Output" pane with a list of actions and their results:

#	Time	Action	Message
165	17:00:48	SELECT position, COUNT(*) AS Total_Employees FR...	7 row(s) returned
166	17:00:48	SELECT position, COUNT(*) AS Total_Employees FR...	1 row(s) returned
167	17:00:48	SELECT * FROM physician ORDER BY name ASC LIMIT...	9 row(s) returned
168	17:00:49	SELECT * FROM physician ORDER BY employeeid DES...	9 row(s) returned
169	17:00:49	SELECT * FROM physician WHERE employeeid IN (S...	3 row(s) returned
170	17:00:49	SELECT * FROM physician WHERE employeeid NOT IN ...	6 row(s) returned

Step 17: Positions Ranked by Employee Count

The screenshot shows a SQL query in a query editor window titled "EMAS". The query is as follows:

```
67 (
68     SELECT head
69     FROM department
70 );
71 SELECT position,
72     COUNT(*) AS Total_Employees
73 FROM physician
74 GROUP BY position
75 ORDER BY Total_Employees DESC;
```

The query results are displayed in a grid with the following columns: position and Total_Employees. The results show 7 rows of data:

position	Total_Employees
Surgical Attending Physician	3
Staff Internist	1
Attending Physician	1
Senior Attending Physician	1
Head Chief of Medicine	1
MD Resident	1
Attending Psychiatrist	1

The bottom of the screenshot shows the "Output" pane with a list of actions and their results:

#	Time	Action	Message
184	17:01:30	SELECT position, COUNT(*) AS Total_Employees FR...	1 row(s) returned
185	17:01:30	SELECT * FROM physician ORDER BY name ASC LIMIT...	9 row(s) returned
186	17:01:30	SELECT * FROM physician ORDER BY employeeid DES...	9 row(s) returned
187	17:01:30	SELECT * FROM physician WHERE employeeid IN (S...	3 row(s) returned
188	17:01:30	SELECT * FROM physician WHERE employeeid NOT IN ...	6 row(s) returned
189	17:01:30	SELECT position, COUNT(*) AS Total_Employees FR...	7 row(s) returned

Step 18: Positions Containing "Attending"

The screenshot shows the SQL Developer interface with a query window titled "Week_2_05". The query is as follows:

```
70 );
71 SELECT position,
72        COUNT(*) AS Total_Employees
73 FROM physician
74 GROUP BY position
75 ORDER BY Total_Employees DESC;
76 SELECT *
77 FROM physician
78 WHERE position LIKE '%Attending%';
```

The "Result Grid" displays the following data:

employeeid	name	position	ssn
2	Elliot Reid	Attending Physician	222222222
3	Christopher Turk	Surgical Attending Physician	333333333
4	Percival Cox	Senior Attending Physician	444444444
6	Todd Quinlan	Surgical Attending Physician	666666666
7	John Wen	Surgical Attending Physician	777777777
9	Molly Clock	Attending Psychiatrist	999999999
NULL	NULL	NULL	NULL

The "Output" pane shows the execution log with the following messages:

#	Time	Action	Message
204	17:02:15	SELECT * FROM physician ORDER BY name ASC LIMIT...	9 row(s) returned
205	17:02:15	SELECT * FROM physician ORDER BY employeeid DES...	9 row(s) returned
206	17:02:15	SELECT * FROM physician WHERE employeeid IN (S...	3 row(s) returned
207	17:02:15	SELECT * FROM physician WHERE employeeid NOT IN ...	6 row(s) returned
208	17:02:15	SELECT position, COUNT(*) AS Total_Employees FR...	7 row(s) returned
209	17:02:15	SELECT * FROM physician WHERE position LIKE "%Atte...	6 row(s) returned

Step 19: Head of Surgery

The screenshot shows the SQL Developer interface with a query window titled "Week_2_05". The query is as follows:

```
72 COUNT(*) AS Total_Employees
73 FROM physician
74 GROUP BY position
75 ORDER BY Total_Employees DESC;
76 SELECT *
77 FROM physician
78 WHERE position LIKE '%Attending%';
79 SELECT p.name
80 FROM physician p
81 JOIN department d
82 ON p.employeeid = d.head
83 WHERE d.name = 'Surgery';
```

The "Result Grid" displays the following data:

name
John Wen

The "Output" pane shows the execution log with the following messages:

#	Time	Action	Message
225	17:02:58	SELECT * FROM physician ORDER BY employeeid DES...	9 row(s) returned
226	17:02:58	SELECT * FROM physician WHERE employeeid IN (S...	3 row(s) returned
227	17:02:58	SELECT * FROM physician WHERE employeeid NOT IN ...	6 row(s) returned
228	17:02:58	SELECT position, COUNT(*) AS Total_Employees FR...	7 row(s) returned
229	17:02:58	SELECT * FROM physician WHERE position LIKE "%Atte...	6 row(s) returned
230	17:02:58	SELECT p.name FROM physician p JOIN department d O...	1 row(s) returned

Step 20: Departments + Heads Alphabetically

The screenshot shows the SQL Developer interface with a query window titled 'Week_2_05'. The query is as follows:

```

78 WHERE position LIKE '%Attending%';
79 SELECT p.name
80 FROM physician p
81 JOIN department d
82 ON p.employeeid = d.head
83 WHERE d.name = 'Surgery';
84 SELECT d.name,
85        p.name
86 FROM department d
87 JOIN physician p
88 ON d.head = p.employeeid
89 ORDER BY d.name;

```

The Result Grid shows the following data:

name	name
General Medicine	Percival Cox
Psychiatry	Molly Clock
Surgery	John Wen

The Action Output pane shows the following log entries:

#	Time	Action	Message
247	17:03:40	SELECT * FROM physician WHERE employeeid IN (S...	3 row(s) returned
248	17:03:40	SELECT * FROM physician WHERE employeeid NOT IN ...	6 row(s) returned
249	17:03:40	SELECT position, COUNT(*) AS Total_Employees FR...	7 row(s) returned
250	17:03:40	SELECT * FROM physician WHERE position LIKE "%Atte...	6 row(s) returned
251	17:03:40	SELECT p.name FROM physician p JOIN department d O...	1 row(s) returned
252	17:03:40	SELECT d.name, p.name FROM department d JOIN ...	3 row(s) returned

Step 21: Physicians Not Involved in Surgery

The screenshot shows the SQL Developer interface with a query window titled 'Week_2_05'. The query is as follows:

```

84 SELECT d.name,
85        p.name
86 FROM department d
87 JOIN physician p
88 ON d.head = p.employeeid
89 ORDER BY d.name;
90 SELECT *
91 FROM physician
92 WHERE position NOT LIKE '%Surgical%';

```

The Result Grid shows the following data:

employeeid	name	position	ssn
1	John Dorian	Staff Internist	111111111
2	Elliot Reid	Attending Physician	222222222
4	Percival Cox	Senior Attending Physician	444444444
5	Bob Kelso	Head Chief of Medicine	555555555
8	Keith Dudemeister	MD Resident	888888888
9	Molly Clock	Attending Psychiatrist	999999999
NULL	NULL	NULL	NULL

The Action Output pane shows the following log entries:

#	Time	Action	Message
270	17:04:29	SELECT * FROM physician WHERE employeeid NOT IN ...	6 row(s) returned
271	17:04:29	SELECT position, COUNT(*) AS Total_Employees FR...	7 row(s) returned
272	17:04:29	SELECT * FROM physician WHERE position LIKE "%Atte...	6 row(s) returned
273	17:04:29	SELECT p.name FROM physician p JOIN department d O...	1 row(s) returned
274	17:04:29	SELECT d.name, p.name FROM department d JOIN ...	3 row(s) returned
275	17:04:29	SELECT * FROM physician WHERE position NOT LIKE '...	6 row(s) returned

Step 22: Positions With At Least 2 Employees

The screenshot shows the SQL Developer interface with a query window titled 'Week_2_05'. The query is as follows:

```
89 ORDER BY d.name;  
90 • SELECT *  
91 FROM physician  
92 WHERE position NOT LIKE '%Surgical%';  
93 • SELECT position,  
94 COUNT(*) AS Total  
95 FROM physician  
96 GROUP BY position  
97 HAVING COUNT(*) >= 2;
```

The 'Result Grid' shows the following data:

position	Total
Surgical Attending Physician	3

The 'Output' pane shows the 'Action Output' with the following messages:

#	Time	Action	Message
291	17:05:16	SELECT * FROM physician ORDER BY employeeid DES...	9 row(s) returned
292	17:05:16	SELECT * FROM physician WHERE employeeid IN (S...	3 row(s) returned
293	17:05:17	SELECT * FROM physician WHERE employeeid NOT IN ...	6 row(s) returned
294	17:05:17	SELECT position, COUNT(*) AS Total_Employees FR...	7 row(s) returned
295	17:05:17	SELECT * FROM physician WHERE position LIKE "%Atte...	6 row(s) returned
296	17:05:17	SELECT p.name FROM physician p JOIN department d O...	1 row(s) returned
297	17:05:17	SELECT d.name, p.name FROM department d JOIN ...	3 row(s) returned
298	17:05:17	SELECT * FROM physician WHERE position NOT LIKE '...	6 row(s) returned
299	17:05:17	SELECT position, COUNT(*) AS Total FROM physi...	1 row(s) returned

Step 23: Departments With Valid Heads

The screenshot shows the SQL Developer interface with a query window titled 'Week_2_05'. The query is as follows:

```
97 HAVING COUNT(*) >= 2;  
98 • SELECT *  
99 FROM department d  
100 WHERE EXISTS  
101 (  
102 SELECT 1  
103 FROM physician p  
104 WHERE p.employeeid = d.head  
105 );
```

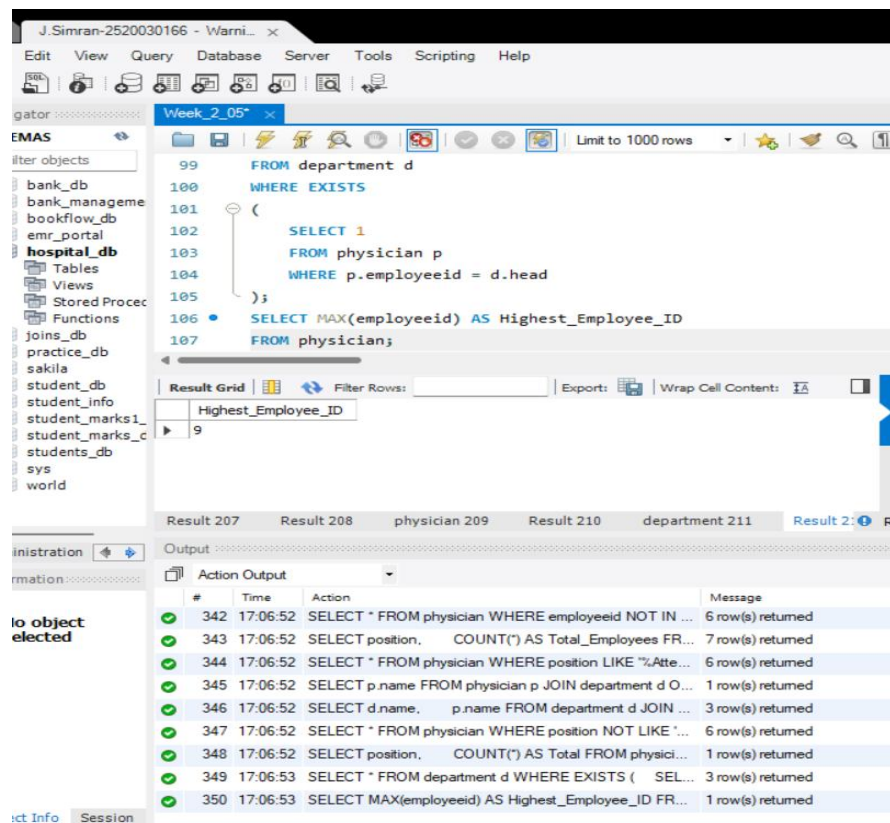
The 'Result Grid' shows the following data:

departmentid	name	head
1	General Medicine	4
2	Surgery	7
3	Psychiatry	9
NULL	NULL	NULL

The 'Output' pane shows the 'Action Output' with the following messages:

#	Time	Action	Message
316	17:06:15	SELECT * FROM physician WHERE employeeid IN (S...	3 row(s) returned
317	17:06:15	SELECT * FROM physician WHERE employeeid NOT IN ...	6 row(s) returned
318	17:06:15	SELECT position, COUNT(*) AS Total_Employees FR...	7 row(s) returned
319	17:06:15	SELECT * FROM physician WHERE position LIKE "%Atte...	6 row(s) returned
320	17:06:15	SELECT p.name FROM physician p JOIN department d O...	1 row(s) returned
321	17:06:15	SELECT d.name, p.name FROM department d JOIN ...	3 row(s) returned
322	17:06:15	SELECT * FROM physician WHERE position NOT LIKE '...	6 row(s) returned
323	17:06:15	SELECT position, COUNT(*) AS Total FROM physi...	1 row(s) returned
324	17:06:15	SELECT * FROM department d WHERE EXISTS (SEL...	3 row(s) returned

Step 24: Highest Employee ID



SQL Developer interface showing the query execution for Step 24: Highest Employee ID.

Query:

```
99 FROM department d
100 WHERE EXISTS
101 (
102     SELECT 1
103     FROM physician p
104     WHERE p.employeeid = d.head
105 );
106 SELECT MAX(employeeid) AS Highest_Employee_ID
107 FROM physician;
```

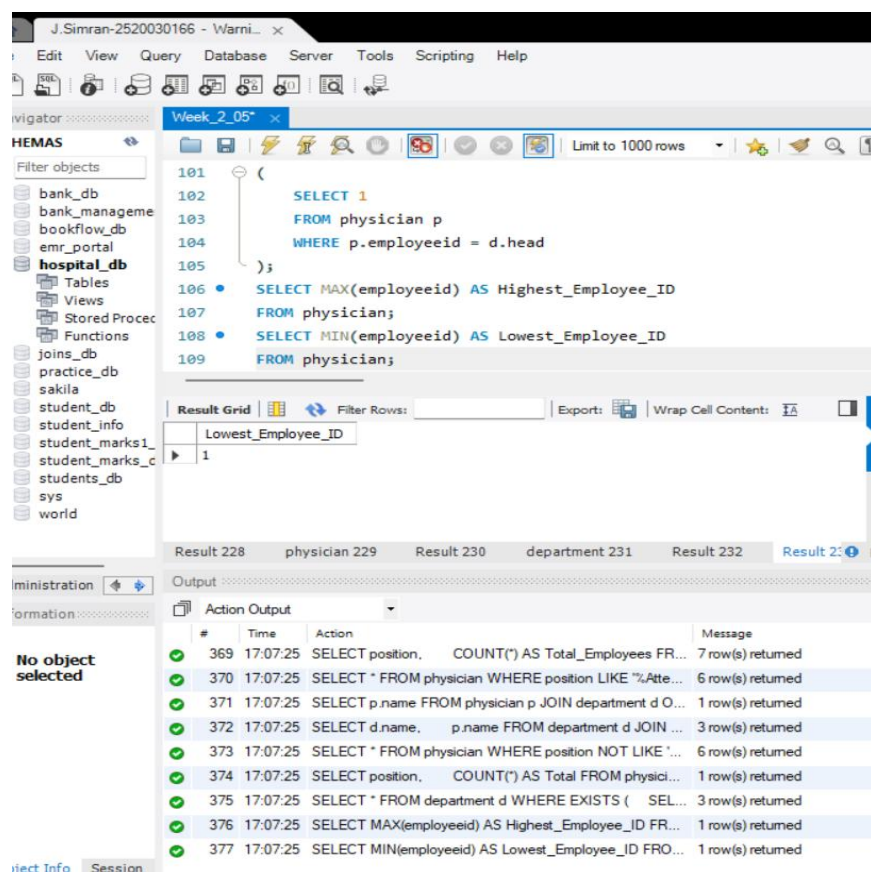
Result Grid:

Column Name	Value
Highest_Employee_ID	9

Output:

#	Time	Action	Message
342	17:06:52	SELECT * FROM physician WHERE employeeid NOT IN ...	6 row(s) returned
343	17:06:52	SELECT position, COUNT(*) AS Total_Employees FR...	7 row(s) returned
344	17:06:52	SELECT * FROM physician WHERE position LIKE "%Atte...	6 row(s) returned
345	17:06:52	SELECT p.name FROM physician p JOIN department d O...	1 row(s) returned
346	17:06:52	SELECT d.name, p.name FROM department d JOIN ...	3 row(s) returned
347	17:06:52	SELECT * FROM physician WHERE position NOT LIKE '...	6 row(s) returned
348	17:06:52	SELECT position, COUNT(*) AS Total FROM physi...	1 row(s) returned
349	17:06:53	SELECT * FROM department d WHERE EXISTS (SEL...	3 row(s) returned
350	17:06:53	SELECT MAX(employeeid) AS Highest_Employee_ID FR...	1 row(s) returned

Step 25: Lowest Employee ID



SQL Developer interface showing the query execution for Step 25: Lowest Employee ID.

Query:

```
101 (
102     SELECT 1
103     FROM physician p
104     WHERE p.employeeid = d.head
105 );
106 SELECT MAX(employeeid) AS Highest_Employee_ID
107 FROM physician;
108 SELECT MIN(employeeid) AS Lowest_Employee_ID
109 FROM physician;
```

Result Grid:

Column Name	Value
Lowest_Employee_ID	1

Output:

#	Time	Action	Message
369	17:07:25	SELECT position, COUNT(*) AS Total_Employees FR...	7 row(s) returned
370	17:07:25	SELECT * FROM physician WHERE position LIKE "%Atte...	6 row(s) returned
371	17:07:25	SELECT p.name FROM physician p JOIN department d O...	1 row(s) returned
372	17:07:25	SELECT d.name, p.name FROM department d JOIN ...	3 row(s) returned
373	17:07:25	SELECT * FROM physician WHERE position NOT LIKE '...	6 row(s) returned
374	17:07:25	SELECT position, COUNT(*) AS Total FROM physi...	1 row(s) returned
375	17:07:25	SELECT * FROM department d WHERE EXISTS (SEL...	3 row(s) returned
376	17:07:25	SELECT MAX(employeeid) AS Highest_Employee_ID FR...	1 row(s) returned
377	17:07:25	SELECT MIN(employeeid) AS Lowest_Employee_ID FRO...	1 row(s) returned